Cross Traffic Management

A Formally Verified, V2I/V2V-Coordinated Autonomous Intersection Management System for a Four-Leg, Single-Lane Crossroad

Technical Report

Formal Model (UPPAAL) · Embedded Software (FreeRTOS/C) · Hardware RTL (VHDL) · Requirements Engineering

Prepared by

Boiddo Sumon
Ferdous Soaib
Nnachi-Egwu Nnaemeka
Oyemade Oluwasholape Daniel

July 12, 2026

This report consolidates the requirements specification, the UPPAAL timed-automata formal model, the FreeRTOS/C embedded implementation, and the VHDL RTL implementation of the Cross Traffic Management autonomous intersection manager (AIM) into a single, traceable technical document.

Contents

1 Introduction

1.1 Motivation

Conventional traffic-light control wastes green time on empty approaches and cannot exploit the fact that many pairs of vehicle movements never actually occupy the same physical space inside an intersection. As autonomous, connected vehicles become realistic, intersection control can be moved from a fixed, time-multiplexed signal phase plan to a *reservation-based* scheme in which vehicles negotiate crossing rights directly with a roadside coordinator. This report documents the design, formal verification, and dual (software and hardware) implementation of such a system: **Cross Traffic Management (CTM)** – a conflict-aware Autonomous Intersection Management (AIM) system for a four-leg, single-lane crossroad, coordinated entirely through Vehicle-to-Infrastructure (V2I) and Vehicle-to-Vehicle (V2V) communication, with no traffic lights and no collisions.

1.2 System Summary

The system removes traffic lights entirely. Every autonomous vehicle (AV) approaching the intersection sends a V2I *crossing request* (direction and intended movement) to a single Intersection Manager (IM), realized as a roadside unit (RSU). The IM checks the request against the crossings that are currently active and either *grants* or *defers* it. A granted vehicle broadcasts its crossing intent over V2V so that neighbouring vehicles remain aware of active crossings, then physically crosses the intersection for a bounded service time before notifying the IM on exit, freeing the shared intersection state. A deferred vehicle waits a bounded interval and retries, guaranteeing that no request is starved.

The same compatibility rule – a variant of an **M/M/2** queuing discipline with two independent "compatible movement" servers plus two additional shareable/exclusive modes – is expressed and checked in three independent artefacts:

1. a **UPPAAL** timed-automata model, formally verified against five safety and liveness properties;
2. a **FreeRTOS/C** embedded software implementation (`aim_freertos.c`) running as a multi-task real-time simulation;
3. a **VHDL** register-transfer-level hardware implementation (`aim_intersection_manager`), verified with a directed testbench in ModelSim/Questa.

1.3 Report Structure

Section 2 introduces the stakeholders and system boundaries. Section 3 describes the intersection topology, actors and use cases. Section 4 states the functional and non-functional requirements and the compatibility rule that underlies all three implementations. Section 5 presents the UPPAAL formal model, and Section 6 its verification results. Sections 7–9 describe the FreeRTOS/C embedded software, its V2I/V2V protocol timing, and runtime simulation evidence. Section 10 describes the VHDL hardware implementation and its testbench. Section 11 formalises the crossing-mode mapping, Section 12 gives end-to-end requirement traceability, Section 13 lists the system's scope and constraints, and Section 15 concludes with key takeaways and future work.

2 System Overview and Stakeholders

The system models cross-traffic management for a four-leg, single-lane crossroad. Autonomous vehicles cross the shared intersection without traffic lights, without collisions, and with minimal waiting time; V2I and V2V communication fully replace traditional signal control.

Table 1 summarises the four stakeholders identified for the system and their role.

Table 1: System stakeholders

Stakeholder	Role
Autonomous Vehicles	Cross the intersection safely and quickly.
Intersection Manager (IM)	Coordinates crossings and prevents conflicts.
Road Infrastructure	Provides the V2I communication point (the RSU hosting the IM).
Other Vehicles	Receive V2V awareness broadcasts of active crossings.

3 Topology, Actors and Use Cases

3.1 Intersection Topology

The intersection has four approaches – North, South, East and West – each with a single entry lane and a single exit lane, following German right-hand traffic conventions. Figure 1 illustrates the layout.

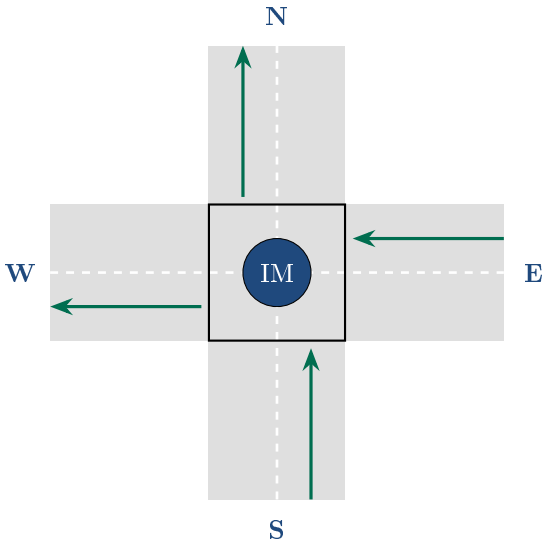

Figure 1: Four-leg, single-lane-per-approach intersection topology. The Intersection Manager (IM) is a logical roadside coordination point at the center; each approach has one entry lane (arrow shown) and one exit lane.

Each vehicle arriving on an approach may perform one of three movements: go *straight*, turn *right*, or turn *left*. With a single lane per approach, all three movements originate from the same lane, so the compatibility rule (Section 4.3) must reason about movement type, not lane assignment.

3.2 Actors

Three actors participate in the system:

- **Autonomous Vehicle (AV)** – arrives, requests a crossing, receives a grant or defer, broadcasts its intent, crosses, and notifies the IM on exit.

- **Intersection Manager (IM / RSU)** – checks compatibility, grants or defers requests, and updates the shared crossing state.
- **Other Vehicles (V2V peers)** – passively receive crossing-intent broadcasts from actively crossing vehicles, maintaining local awareness of the intersection state.

3.3 Use Case Diagram

Figure 2 shows the ten use cases (UC1–UC10) that the three actors participate in. The full use case diagram is provided in the accompanying Requirements Specification document; it is reproduced here for traceability.

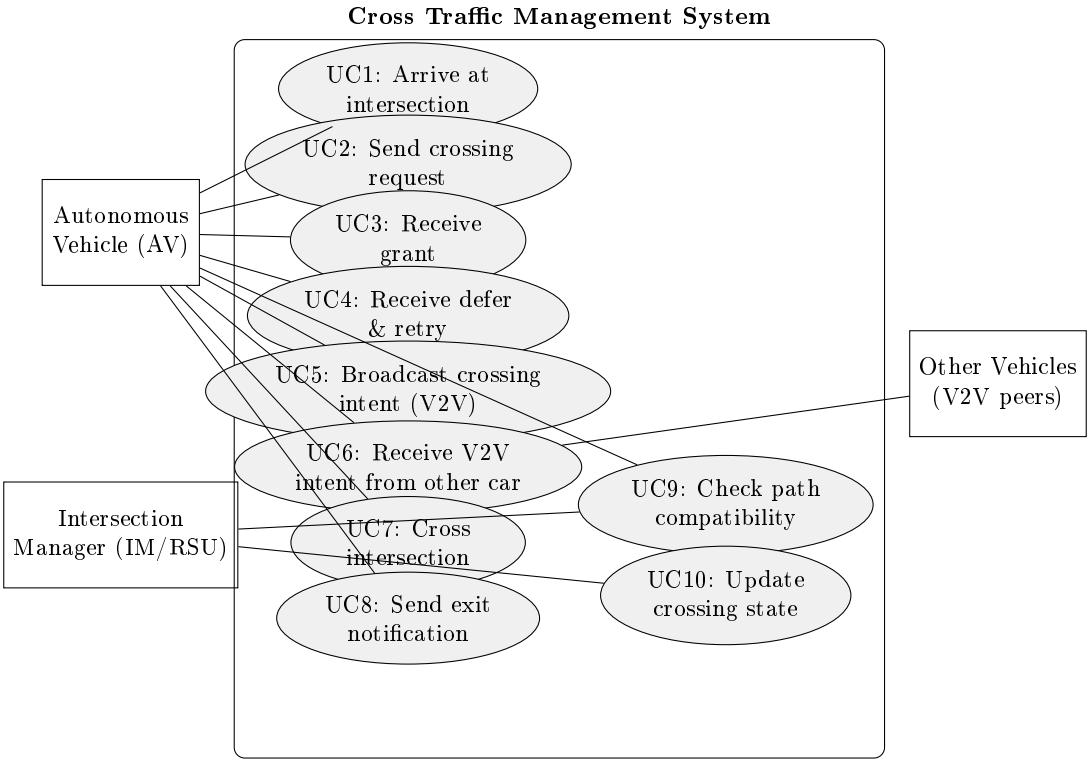

Figure 2: Use case diagram: three actors and ten coordinated use cases.

4 Requirements Specification

4.1 Functional Requirements

Table 2 lists the ten functional requirements (FR1–FR10) that define what the system shall do.

Table 2: Functional Requirements

ID	Requirement
FR1	Intersection topology: 4 approaches (N/S/E/W), one entry and one exit lane each.
FR2	Movement types: straight, right turn, or left turn per vehicle.
FR3	V2I request: the vehicle sends its direction and intended movement to the IM.
FR4	Compatibility check: the IM checks the request against currently active crossings.
FR5	Grant: the IM grants the request when compatible; the vehicle must wait for the grant.
FR6	Defer: the IM defers the request on conflict; the vehicle waits, then retries.
FR7	V2V broadcast: the vehicle broadcasts its crossing intent immediately after being granted.
FR8	Crossing: the vehicle crosses the intersection within the defined service time.
FR9	Exit notification: the vehicle notifies the IM on completion so the crossing state can be freed.
FR10	No traffic lights: all coordination is via V2I reservation plus V2V awareness.

4.2 Non-Functional Requirements

Table 3 lists the non-functional requirements governing *how well* the system must perform. The IDs NFR1–NFR7 are assigned here in the order the properties are presented and are used consistently for traceability in Section 12.[1]

Table 3: Non-Functional Requirements

ID	Property	Statement
NFR1	Safety	No conflicting crossings, ever.
NFR2	Liveness	Every request is eventually granted.
NFR3	Deadlock-free	The IM can always respond.
NFR4	Bounded	Maximum of 4 vehicles crossing at once.
NFR5	Throughput (M/M/2)	Two compatible movement streams may be served at once.
NFR6	Real-time	Grant/defer decisions are made in a single (atomic) step.
NFR7	Protocol format	Every message carries direction, movement, and message type.

4.3 Compatibility Rule (M/M/2)

The central design decision of the system is the *compatibility rule* that the IM applies to decide whether a newly requested movement can be granted alongside the movement(s) currently active. It is modelled as an M/M/2-style queuing discipline (Markovian arrivals, exponential service

[1] The source presentation lists these seven properties without explicit numeric labels on the summary slide, but individual slides elsewhere in the deck refer to "NFR1" (safety), "NFR2" (liveness) and "NFR6" (real-time response). The numbering adopted here is consistent with every such reference found in the source material.

time, two independent "servers", i.e. two disjoint classes of straight-through traffic that never geometrically cross), extended with a fully shareable right-turn mode and a fully exclusive left-turn mode. Table 4 gives representative scenarios.

Table 4: Compatibility rule – representative scenarios

Scenario	OK?	Reason
N-straight + S-straight	Yes	Opposite lanes, paths never cross (M/M/2 server 1).
E-straight + W-straight	Yes	Opposite lanes, paths never cross (M/M/2 server 2).
Any right turns together	Yes	Each right turn stays within its own corner of the intersection.
N-straight + E-straight	No	Paths cross in the center of the intersection.
Any left turn + anything	No	Left turns sweep through the center and conflict with every other movement.

5 UPPAAL Formal Model

The behaviour of the system is formally modelled in **UPPAAL** (captured from UPPAAL 5.1.0-beta5, model file `aim_uppaal_final.xml`) using three timed-automata templates coordinated through typed channels, as summarised in Table 5.

Table 5: UPPAAL templates

Template	Instances	Description
`IM`	1	Ready state plus four committed `Proc` states; grants or defers each of the four requesting directions.
`Car(id)`	4 (N, S, E, W)	`Idle` → `Queued` → `WaitResp` → `Deferred` / `V2V` → `Crossing` → `Idle`.
`V2VListener(sid)`	4 (N, S, E, W)	Single `Listen` state; reacts to the broadcast channel `vv[sid]?`.

5.1 Shared Declarations

The global (shared) declarations coordinating the templates are:

```
int cc; // crossCount - vehicles currently crossing
int cm; // crossMode - which movement class currently owns the intersection
int rm[4]; // requested movement per direction
chan rq[4], gr[4], df[4]; // per-direction request / grant / defer channels
broadcast chan vv[4]; // V2V crossing-intent broadcast channels
```

5.2 Car Protocol State Machine

Figure 3 shows the `Car(id)` automaton, which is mirrored exactly by the `LaneTask` retry loop in the FreeRTOS implementation (Section 7).

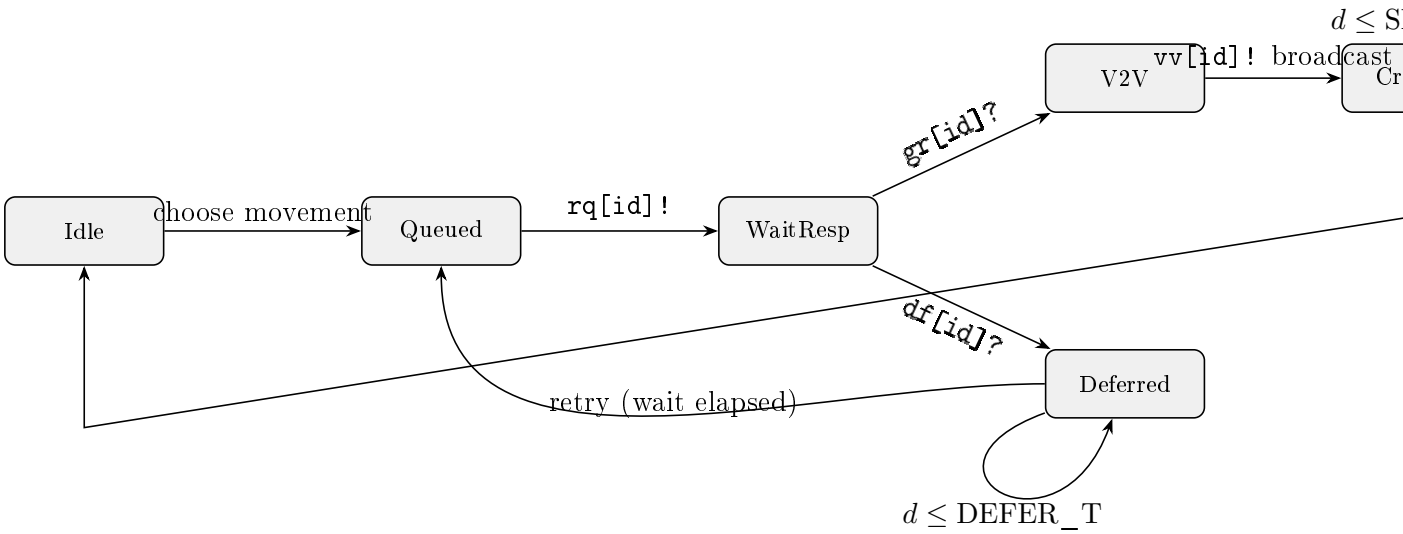

Figure 3: Car(id) protocol state machine, as modelled in UPPAAL. The same six states map one-to-one onto the LaneTask retry loop, the `xQueueSend`/`xQueueReceive` calls, and the `vTaskDelay(SERVICE_MS)` / `vTaskDelay(DEFER_MS)` calls in `aim_freertos.c`.

5.3 Intersection Manager (IM) Template

The IM template holds a `Ready` state and four committed `Proc0`–`Proc3` states, one per requesting direction. Each `Proc_i` fires `gr[i]!` when the requested movement is compatible with the currently active crossing mode, or `df[i]!` otherwise. This mirrors `canGrant()` in the FreeRTOS implementation and `can_grant_fn()` in the VHDL implementation exactly (Table 12). Figure 4 shows the modelled automaton as captured directly from the UPPAAL editor.

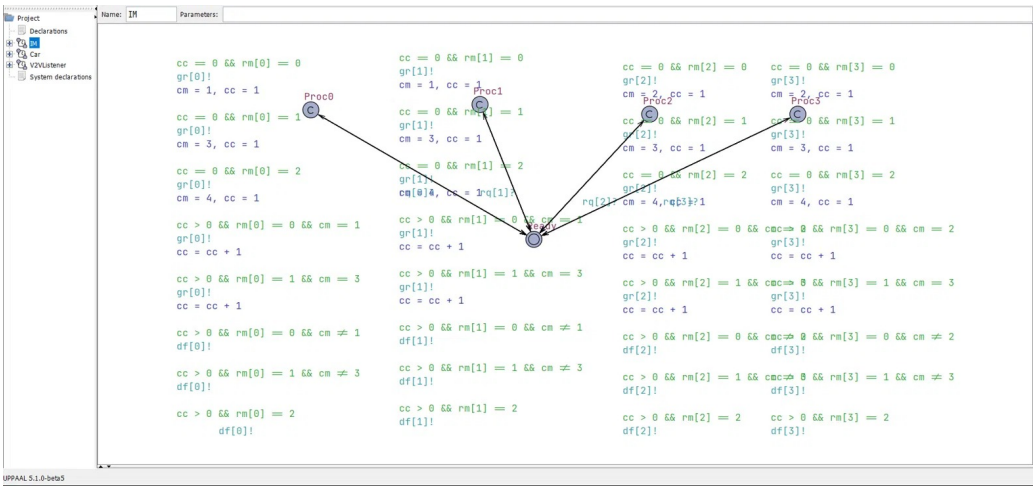

Figure 4: UPPAAL editor view of the Intersection Manager (IM) template: committed `Proc0`-`Proc3` states implement the compatibility guard for each direction. Captured from UPPAAL 5.1.0-beta5 (`aim_uppaal_final.xml`).

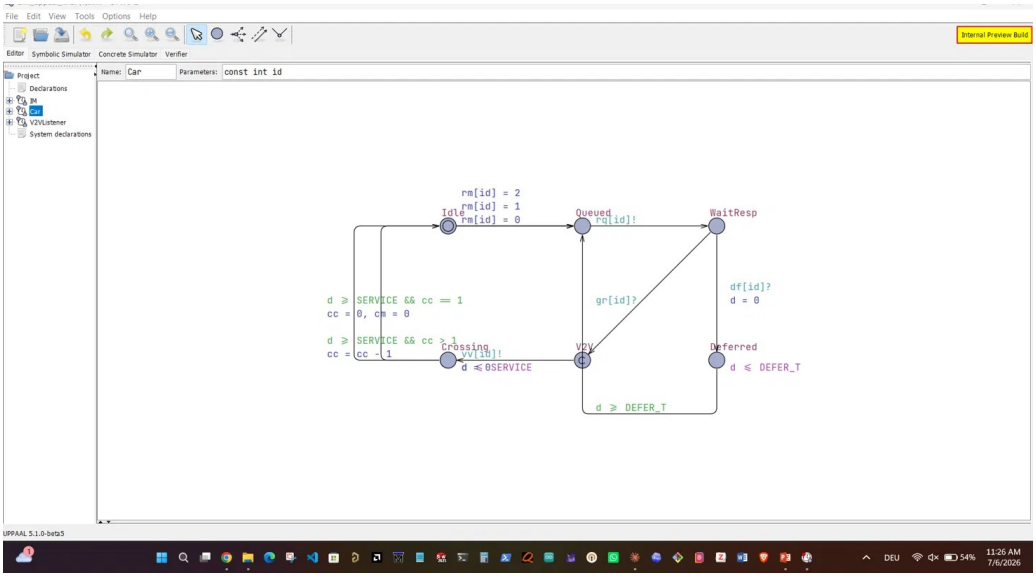

Figure 5: UPPAAL editor view of the `Car(id)` template, corresponding to the state machine in Figure 3.

5.4 V2VListener Template

The `V2VListener(sid)` template has a single `Listen` state and reacts to the broadcast channel `vv[sid]?`, modelling how every non-crossing vehicle passively updates its local awareness whenever any other vehicle broadcasts a crossing intent.

6 Formal Verification Results

Five UPPAAL queries (Q1–Q5) check the safety- and liveness-critical properties of the model. Table 6 lists each query, its formula, and its interpretation; all five are verified **TRUE** against the model, giving formal assurance that the same rules subsequently implemented in FreeRTOS and VHDL are safe by construction.

Table 6: UPPAAL verification queries and results

ID	Property	Formula	Interpretation	Result
Q1	No deadlock	`A[] not deadlock`	The system can always make progress.	TRUE
Q2	Path exclusivity	N-straight & E-straight never together	Conflicting paths never overlap.	TRUE
Q3	M/M/2 server 1	N & S cross together only if both straight	Opposite-lane pairing enforced.	TRUE
Q4	M/M/2 server 2	E & W cross together only if both straight	Opposite-lane pairing enforced.	TRUE
Q5	Bounded crossings	`cc <= 4` at all times	Never more than 4 vehicles crossing.	TRUE

Figure 6 shows the queries as entered in the UPPAAL Verifier, alongside the Car template automaton for reference.

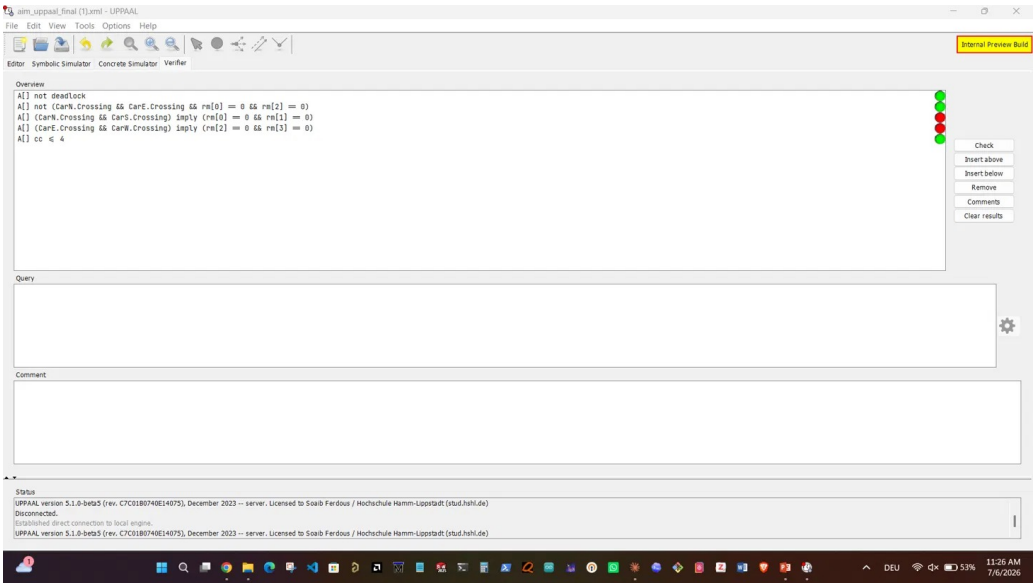

Figure 6: UPPAAL Verifier: all five safety and liveness queries (`A[] not deadlock`, path exclusivity, M/M/2 server pairing Q3/Q4, and `cc <= 4`), each checked against the model and reported as satisfied.

7 FreeRTOS Embedded Software Architecture

The embedded software implementation (`aim_freertos.c`) realises the UPPAAL model as a direct task-for-template mapping running under FreeRTOS. Table 7 lists the task set.

Table 7: FreeRTOS task set

Task	Instances	Priority	Responsibility
`IMTask`	1	+3 (highest)	Reads `v2i_reqQueue`; runs `canGrant()`; sends `GRANT`/`DEFER`; handles `EXIT`.
`LaneTask`	4	+1	Arrival timing; V2I request/retry loop; V2V broadcast; crossing delay.
`V2VListenerTask`	4	+2	Subscribes to `v2v_broadcastQueue[dir]`; updates the local awareness table.

7.1 Queues and Synchronization

- `v2i_reqQueue` – all four lanes' requests to the IM (shared request channel).
- `v2i_respQueue[4]` – IM to `lane[dir]` (GRANT / DEFER).
- `v2v_broadcastQueue[4]` – one queue per sender, fanned out to listeners.
- `imMutex`, `v2vMutex` – protect `crossCount`/`crossMode` and the awareness table, respectively.

7.2 Design Rationale

`IMTask` runs at the highest priority so that it never blocks a `LaneTask` waiting on a grant/defer decision – this directly satisfies NFR6 (real-time response): the IM always decides within one atomic step.

8 V2I / V2V Protocol Flow

Figure 7 traces one complete crossing cycle, from vehicle arrival to exit notification, as implemented in `aim_freertos.c`.

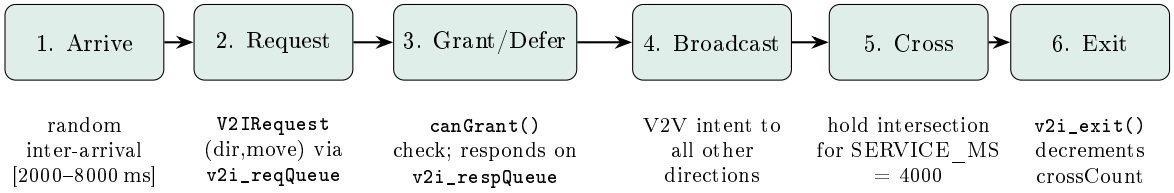

Figure 7: One crossing cycle, from arrival to exit notification, as implemented in `aim_freertos.c`. On DEFER, the vehicle waits `DEFER_MS` = 3000 ms then re-enters step 2 (retry), guaranteeing liveness (NFR2) without polling the IM continuously.

Table 8 lists the key timing constants used by the simulator.

Table 8: Key timing constants (`aim_freertos.c`)

Constant	Value	Meaning
`MIN_ARR_MS` / `MAX_ARR_MS`	2000 / 8000	Vehicle inter-arrival window (ms).
`SERVICE_MS`	4000	Time spent crossing the intersection (ms).
`DEFER_MS`	3000	Wait time before retrying after a defer (ms).
`MAX_QUEUE`	6	Max queued vehicles per approach.

9 Runtime Simulation Results

The FreeRTOS AIM simulator's `canGrant()` function was exercised against four representative cases, each captured directly from the simulator console.

9.1 Case 1: Compatible Crossings – N-straight + S-straight

Both cars request the same mode (`MODE_NS_STR`) and are both `GRANT`ed; `crossMode` becomes `NS_STRAIGHT`. This exercises M/M/2 server 1 (opposite lanes never cross).

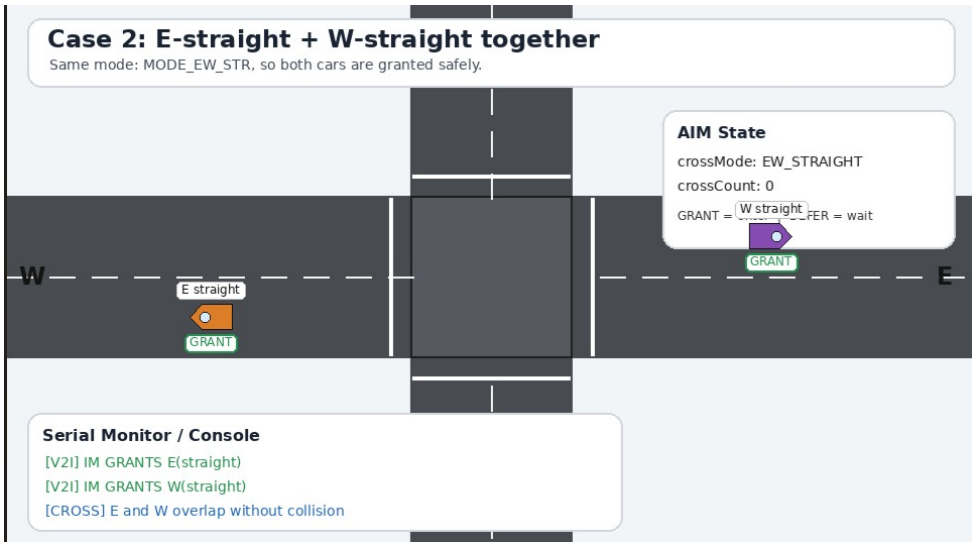

Figure 8: FreeRTOS AIM simulator console, Case 1: N-straight + S-straight, both GRANTed.

9.2 Case 2: Compatible Crossings – E-straight + W-straight

Both cars again request the same mode (`MODE_EW_STR`) and are both `GRANT`ed; `crossMode` becomes `EW_STRAIGHT`, with `crossCount` = 0 before entry. This exercises M/M/2 server 2.

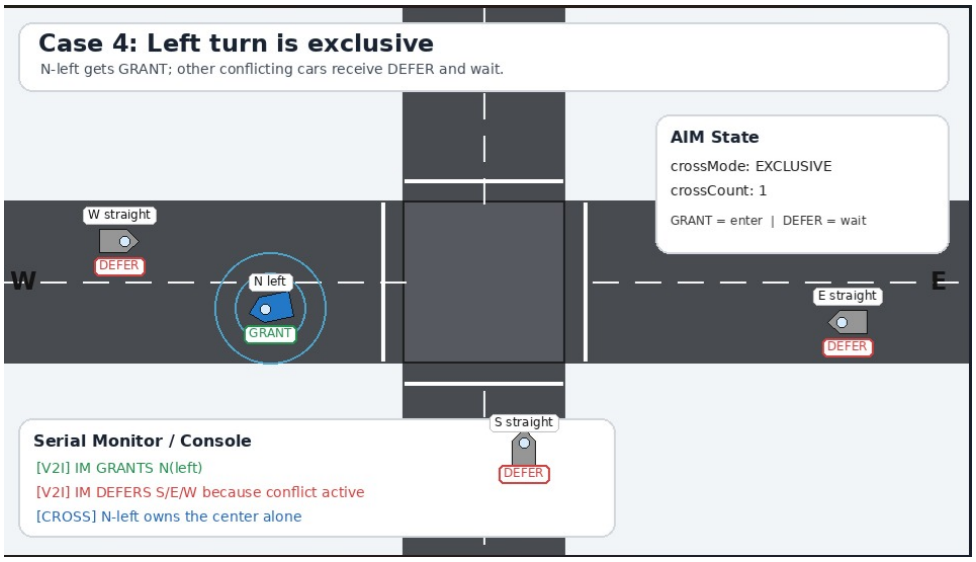

Figure 9: FreeRTOS AIM simulator console, Case 2: E-straight + W-straight, both GRANTed.

9.3 Case 3: All Right Turns Together

`MODE_RTURN` allows all four corners to be used at once: `crossMode` = `RIGHT_TURN`, `crossCount` = 4, all four requests GRANTed simultaneously.

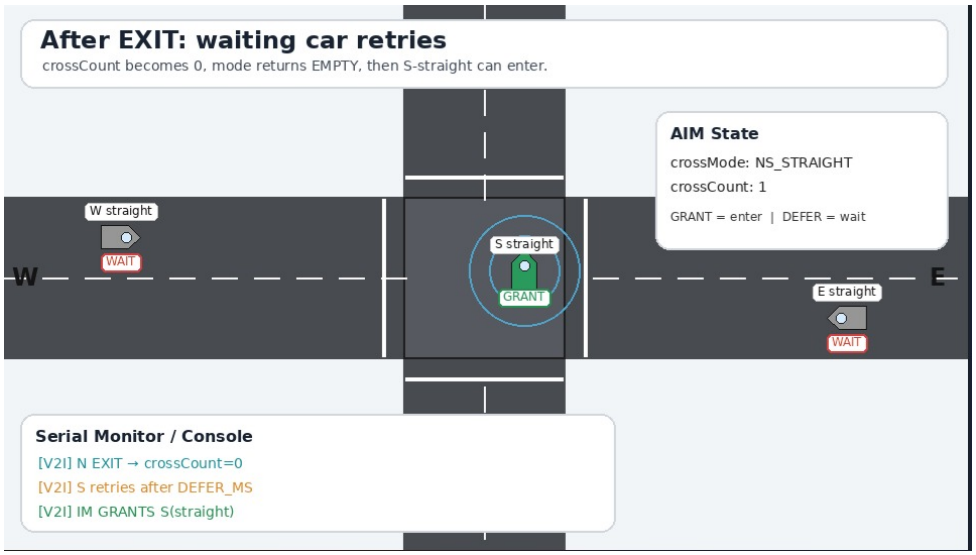

Figure 10: FreeRTOS AIM simulator console, Case 3: all four right turns GRANTed together.

9.4 Case 4: Exclusive Left Turn and Retry

The N-left request is GRANTed alone (`MODE_EXCL`); the S/E/W straight requests all DEFER until it exits. Once `crossCount` returns to 0 on EXIT, the deferred car retries after `DEFER_MS` and is subsequently GRANTed. This demonstrates NFR2 (liveness): every deferred vehicle is eventually granted, with no starvation.

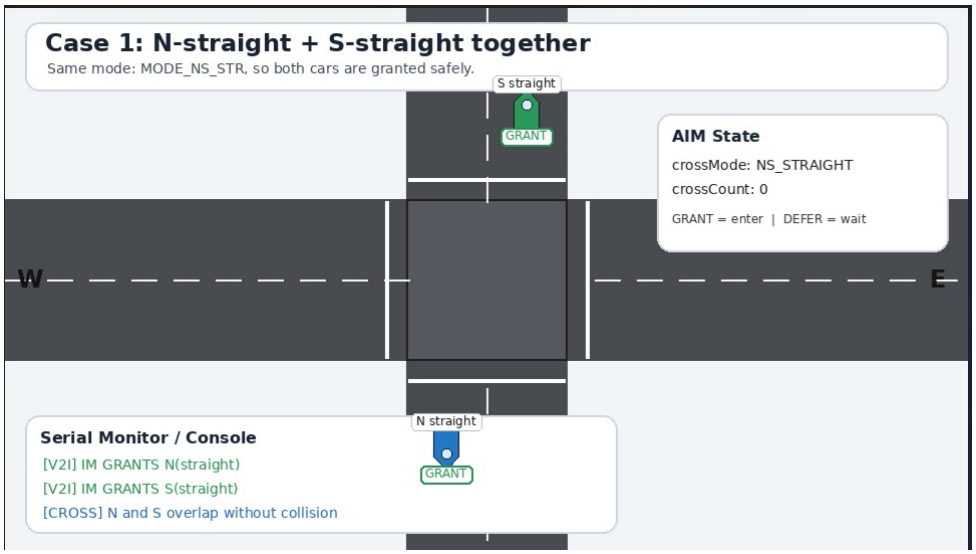

Figure 11: FreeRTOS AIM simulator console, Case 4: exclusive left turn, followed by the deferred car's retry and grant.

10 VHDL Hardware Implementation

The same compatibility rule is synthesised as RTL in the entity `aim_intersection_manager`. Table 9 lists its key ports.

Table 9: Key ports of `aim_intersection_manager`

Port		Description
`clk`, `rst`		System clock; synchronous, active-high reset.
`v2i_req_valid[4]`, `v2i_req_move[8]`		1 bit + 2 bits per car – request valid + movement encoding (`00` straight, `01` right, `10` left).
`v2i_grant[4]`, `v2i_defer[4]`		One-cycle pulse per car – grant or defer.
`v2i_exit[4]`		Car has finished crossing.
`v2v_intent[4]`	→	V2V broadcast passthrough.
`v2v_aware[4]`		
`cross_count[3]`, `cross_mode[2]`		`cnt_reg` (0–7) and `mode_reg` (`"00"`..`"11"`).

10.1 Compatibility Function `can_grant_fn`

The grant decision is computed by `can_grant_fn(dir, mv, mode_r, excl, cnt)`, which evaluates as follows:

- `cnt = 0` → **true** (intersection empty).
- `excl = '1'` → **false** (an exclusive left-turn crossing is active).
- `mv = LEFT` → **false** (left turns are always exclusive).
- `get_mode_fn(dir, mv) = mode_r` → **true** (requested movement matches the currently active shareable mode).
- otherwise → **false** (defer).

This is identical, decision-for-decision, to `canGrant()` in the FreeRTOS implementation and to the `Proc_i` guards in the UPPAAL `IM` template.

10.2 Testbench

The design is verified with the directed testbench `tb_aim_intersection_manager` in ModelSim/Questa, running at 100 MHz (10 ns clock period). Table 10 lists the eight directed test steps.

Table 10: Testbench steps (`tb_aim_intersection_manager`)

Step	Behaviour
T1–T2	N then S straight, intersection empty → both GRANT; M/M/2 server 1 (mode `01`).
T3	E straight while N/S active → DEFER (cross-conflict).
T4	N & S exit → `cross_count` = 0, mode reset to empty.
T5	N & W broadcast V2V intent → `v2v_aware` reflects both.
T6	All four right turns at once → all GRANT, mode = `11` (right-turn).
T7–T8	N left turn → GRANT + exclusive; S straight during it → DEFER.

11 M/M/2 Crossing-Mode Mapping

The functions `getMode()`/`canGrant()` in C and `can_grant_fn()` in VHDL enforce an identical five-mode rule, summarised in Table 11.

Table 11: M/M/2 crossing-mode mapping

Mode	Meaning	VHDL encoding	Compatible movements
`MODE_EMPTY`	Intersection idle	`"00"`	Any new request accepted.
`MODE_NS_STR` (server 1)	N/S straight active	`"01"`	Only more N/S straight requests.
`MODE_EW_STR` (server 2)	E/W straight active	`"10"`	Only more E/W straight requests.
`MODE_RTURN`	Right turns active	`"11"`	All right turns, any direction.
`MODE_EXCL`	Left turn active	`excl_flag='1'`	No other movement until it exits.

`canGrant()` / `can_grant_fn()`: the request is granted if the intersection is empty, or if the requested movement's mode matches the currently active mode and is shareable (right turns, or the same straight-through server). Left turns and cross-conflicting straight movements are always deferred – this logic is identical in C and VHDL.

12 Traceability: Requirement to Implementation

Table 12 traces each requirement through all three artefacts, showing that the same behaviour is expressed – and independently verified – three ways: 5 UPPAAL queries, the `aim_freertos.c` retry/priority design, and 8 directed VHDL testbench cases (`tb_aim_intersection_manager`).

Table 12: Requirement-to-implementation traceability matrix

Requirement	UPPAAL	FreeRTOS	VHDL
FR4 – compatibility	Guards on `Proc` transitions	`canGrant()` function	`can_grant_fn()` in RTL
FR5/FR6 – grant/defer	`gr[id]!` / `df[id]!`	`RESP_GRANT` / `RESP_DEFER`	`v2i_grant` / `v2i_defer` pulses
FR7 – V2V broadcast	`vv[id]!` broadcast chan.	`v2v_broadcastQueue`	`v2v_intent` input bus
NFR1 – safety	Q2 verified	`canGrant()` prevents conflict	`can_grant_fn()` hardware
NFR4 – bounded (≤ 4)	Q5 verified	`cc <= 4` max crossings	`cnt_reg` range 0–7
NFR5 – M/M/2	`cm` modes 1 and 2	`MODE_NS_STR` / `MODE_EW_STR`	`mode_reg "01"` / `"10"`

13 System Constraints and Scope

Table 13 states the boundaries within which the design operates (SC1–SC7).

Table 13: System constraints and scope

ID	Constraint
SC1	Single-lane approaches – no dedicated turn lanes.
SC2	All vehicles are V2I / V2V capable.
SC3	The Intersection Manager is a single roadside unit (RSU).
SC4	Queuing model: M/M/2 (Markovian arrivals, exponential service, 2 servers).
SC5	Hardware implementation: VHDL (`aim_intersection_manager`), verified in ModelSim/Questa.
SC6	Software implementation: C with FreeRTOS.
SC7	Formal model: UPPAAL timed automata.

14 Discussion and Key Takeaways

One rule, three implementations – safety by construction.
The same `canGrant()` compatibility rule is enforced identically in UPPAAL, C/FreeRTOS, and VHDL, so safety arguments proven at the model level carry over to both the software and hardware realisations by construction rather than by post-hoc testing alone.

Formally verified.
Five UPPAAL queries confirm no deadlock, path exclusivity, correct M/M/2 pairing, and a bounded crossing count.

Liveness guaranteed.
The defer-and-retry loop ensures every vehicle is eventually granted – no starvation.

No traffic lights needed.
V2I reservation plus V2V awareness fully replace signal-phase control.

15 Conclusion and Future Work

This report has presented Cross Traffic Management, a conflict-aware autonomous intersection management system for a four-leg, single-lane crossroad. Starting from a stakeholder analysis and ten functional / seven non-functional requirements, the system's central compatibility rule was formalised as an M/M/2-style queuing discipline and modelled as three cooperating UPPAAL timed automata (`IM`, `Car(id)`, `V2VListener(sid)`). Five verification queries formally establish deadlock-freedom, path exclusivity, correct M/M/2 server pairing, and a bounded crossing count. The identical decision logic was then independently realised in a FreeRTOS/C multi-task embedded implementation and synthesised as VHDL RTL, each validated with its own evidence (runtime simulation traces and a directed ModelSim/Questa testbench, respectively). The end-to-end traceability matrix (Table 12) demonstrates that the safety and liveness properties proven at the formal-model level are preserved, decision-for-decision, all the way down to the synthesizable hardware.

Natural extensions of this work include: relaxing the single-lane-per-approach constraint (SC1) to multiple lanes per approach, which would require re-deriving the conflict-point geometry and moving from two to more queuing servers; replacing the single RSU (SC3) with a distributed or redundant IM for fault tolerance; and closing the loop between the VHDL RTL and a physical or FPGA-in-the-loop testbed to validate real V2I/V2V timing (SAE J2735 message framing) rather than the current simulated timing constants.

References

[1] Nasr, M. S. et al. (2020). *Formal Verification of Heuristic Autonomous Intersection Management Using Statistical Model Checking.* Sensors, MDPI.

[2] SAE International. *SAE J2735 – Dedicated Short Range Communications (DSRC) Message Set Dictionary.*

[3] Behrmann, G., David, A., & Larsen, K. G. *A Tutorial on UPPAAL.* Uppsala University and Aalborg University.

[4] Real Time Engineers Ltd. *FreeRTOS Reference Manual.*